

BÁO CÁO BÀI TẬP LẬP TRÌNH MẠNG

Họ tên: Hoàng Tuấn Minh

MSSV: 20235376

Mã lớp: 168503

Link bài tập: [hoangtminh/Lap-trinh-mang](https://hoangtminh.github.io/Lap-trinh-mang)

1. Bài tập telnet_server

```
fork > C telnet_server.c > main()
1  #define _POSIX_C_SOURCE 200809L
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <sys/socket.h>
6  #include <arpa/inet.h>
7  #include <unistd.h>
8  #include <sys/wait.h>
9  #include <signal.h>
10 #include <errno.h>
11
12 #define PORT 8082
13
14 int check_login(char *user, char *pass) {
15     FILE *f = fopen("db.txt", "r");
16     if (!f) {
17         printf("Khong mo duoc file db.txt\n");
18         return 0;
19     }
20     char u[50], p[50];
21     while (fscanf(f, "%s %s", u, p) != EOF) {
22         if (strcmp(user, u) == 0 && strcmp(pass, p) == 0) {
23             fclose(f);
24             return 1;
25         }
26     }
27     fclose(f);
28     return 0;
29 }
30
31 void sigchld_handler(int s) {
32     int saved_errno = errno;
33     while(waitpid(-1, NULL, WNOHANG) > 0);
34     errno = saved_errno;
```

Code telnet_server 1

```

fork > C telnet_server.c > main()
37 int main() {
38     int listener = socket(AF_INET, SOCK_STREAM, 0);
39     struct sockaddr_in addr = {0};
40     addr.sin_family = AF_INET;
41     addr.sin_addr.s_addr = htonl(INADDR_ANY);
42     addr.sin_port = htons(PORT);
43
44     if (bind(listener, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
45         perror("bind() failed");
46         return 1;
47     }
48     listen(listener, 10);
49     printf("Telnet Server (Multiprocessing) dang chay tren port %d...\n", PORT);
50
51     struct sigaction sa;
52     sa.sa_handler = sigchld_handler;
53     sigemptyset(&sa.sa_mask);
54     sa.sa_flags = SA_RESTART;
55     if (sigaction(SIGCHLD, &sa, NULL) == -1) {
56         perror("sigaction failed");
57         return 1;
58     }
59
60     while (1) {
61         struct sockaddr_in client_addr;
62         socklen_t client_len = sizeof(client_addr);
63         int client_fd = accept(listener, (struct sockaddr *)&client_addr, &client_len);
64
65         if (client_fd < 0) {
66             if (errno == EINTR) continue;
67             perror("accept() failed");
68             continue;
69         }
70

```

Code telnet_server 2

```

fork > C telnet_server.c > main()
71 char client_ip[INET_ADDRSTRLEN];
72 inet_ntop(AF_INET, &client_addr.sin_addr, client_ip, INET_ADDRSTRLEN);
73 printf("Client moi ket noi tu IP: %s (FD: %d)\n", client_ip, client_fd);
74
75 if (fork() == 0) {
76     close(listener);
77
78     send(client_fd, "User Pass: ", 11, 0);
79     int authenticated = 0;
80     char buf[256];
81
82     while (1) {
83         int ret = recv(client_fd, buf, sizeof(buf) - 1, 0);
84         if (ret <= 0) break;
85
86         buf[ret] = 0;
87         while (ret > 0 && (buf[ret-1] == '\n' || buf[ret-1] == '\r')) {
88             buf[--ret] = 0;
89         }
90
91         if (!authenticated) {
92             char user[50], pass[50];
93             if (sscanf(buf, "%s %s", user, pass) == 2 && check_login(user, pass)) {
94                 authenticated = 1;
95                 printf("[%d] Client %s dang nhap thanh cong\n", getpid(), client_ip);
96                 send(client_fd, "Login success. Enter command: ", 30, 0);
97             } else {
98                 send(client_fd, "Login failed. Try again: ", 25, 0);
99             }
100         } else {
101             printf("[%d] COMMAND tu %s: %s\n", getpid(), client_ip, buf);
102
103             char out_file[64];
104             sprintf(out_file, "out_%d.txt", getpid());

```

Code telnet_server 3

```

fork > C telnet_server.c > main()
105
106     char cmd[512];
107     sprintf(cmd, "%s > %s 2>&1", buf, out_file);
108     system(cmd);
109
110     FILE *f = fopen(out_file, "r");
111     if (f) {
112         char result[2048];
113         size_t bytes_read = fread(result, 1, sizeof(result) - 1, f);
114         fclose(f);
115         remove(out_file);
116
117         if (bytes_read > 0) {
118             send(client_fd, result, bytes_read, 0);
119         } else {
120             send(client_fd, "Command executed.\n", 18, 0);
121         }
122     }
123 }
124
125 printf("Client %s (PID: %d) da thoat\n", client_ip, getpid());
126 close(client_fd);
127 exit(0);
128 }
129
130 close(client_fd);
131 }
132
133 return 0;
134 }

```

Code telnet_server 4

- Kết quả:

The screenshot shows the VS Code editor with the `telnet_server.c` file open. The code defines a port of 8082 and includes a `check_login` function that checks a password against a file `db.txt`. The terminal window shows the output of the server and client. The server is running on port 8082, and the client is connecting to it. The client sends a password, and the server responds with the login status. The client then sends a command, and the server executes it.

```

trinh mang/forks$ gcc telnet_client.c -o telnet
thedemo@Demo-VM:~/all app folder/Code/c-c++/lap...$ ./telnet client
User Pass: 123
Login failed. Try again: thedemo23
Login failed. Try again: guest
Login failed. Try again: admin admin
Login success. Enter command: hello
sh: 1: hello: not found

```

Kết quả telnet_server

2. Bài tập http_server

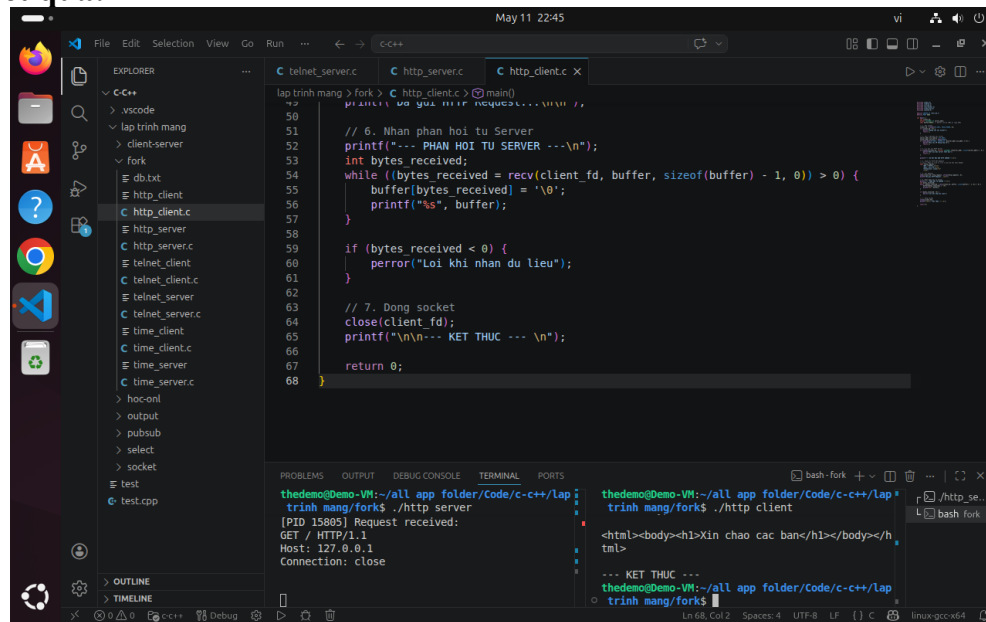
```
fork > C http_server.c > NUM_CHILDREN
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/socket.h>
5  #include <arpa/inet.h>
6  #include <unistd.h>
7  #include <sys/wait.h>
8
9  #define PORT 8080
10 #define NUM_CHILDREN 8
11
12 void handle_http_request(int client) {
13     char buf[2048];
14     int ret = recv(client, buf, sizeof(buf) - 1, 0);
15     if (ret > 0) {
16         buf[ret] = 0;
17         printf("[PID %d] Request received:\n%s\n", getpid(), buf);
18
19         char *msg = "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n<html><body><h1>Xin chao cac";
20         send(client, msg, strlen(msg), 0);
21     }
22     close(client);
23 }
24
25 int main() {
26     int listener = socket(AF_INET, SOCK_STREAM, 0);
27
28     struct sockaddr_in addr;
29     addr.sin_family = AF_INET;
30     addr.sin_addr.s_addr = htonl(INADDR_ANY);
31     addr.sin_port = htons(PORT);
32
33     if (bind(listener, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
34         perror("bind() failed");
```

Code telnet server 1

```
33     if (bind(listener, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
34         perror("bind() failed");
35         return 1;
36     }
37
38     listen(listener, 20);
39     printf("HTTP Preforking Server đang chạy trên port %d với %d tiến trình con...\n", PORT, NUM_CHI
40
41     for (int i = 0; i < NUM_CHILDREN; i++) {
42         pid_t pid = fork();
43
44         if (pid == 0) {
45             while (1) {
46                 int client = accept(listener, NULL, NULL);
47                 if (client < 0) continue;
48
49                 handle_http_request(client);
50             }
51             exit(0);
52         }
53     }
54
55     while (wait(NULL) > 0);
56
57     close(listener);
58     return 0;
59 }
```

Code telnet_server 2

- Kết quả:



The screenshot shows the Visual Studio Code editor with the file `http_server.c` open. The code is a C program that implements a simple HTTP server. It includes headers for `stdio.h`, `stdlib.h`, `string.h`, `sys/socket.h`, `arpa/inet.h`, `unistd.h`, `time.h`, `sys/wait.h`, and `signal.h`. It defines a port `9001` and implements a `sigchld_handler` function to handle child processes. The `get_formatted_time` function formats the current time into a string. The `main` function sets up the server, listens for connections, and handles incoming requests. The terminal output shows the server running and receiving a request from a client.

```
lap_trinh_mang > fork > C http_client.c > main()
printf("Xin gửi http request...\n");

// 6. Nhận phản hồi từ Server
printf("--- PHAN HOI TU SERVER ---\n");
int bytes_received;
while ((bytes_received = recv(client_fd, buffer, sizeof(buffer) - 1, 0)) > 0) {
    buffer[bytes_received] = '\0';
    printf("%s", buffer);
}

if (bytes_received < 0) {
    perror("Loi khi nhan du lieu");
}

// 7. Dong socket
close(client_fd);
printf("\n\n--- KET THUC --- \n");

return 0;
```

thededemo@Demo-VH:~/all app folder/Code/c-c++/lap_trinh_mang/fork\$./http_server

[PID 15805] Request received:

GET / HTTP/1.1

Host: 127.0.0.1

Connection: close

<html><body><h1>Xin chao cac ban</h1></body></html>

--- KET THUC ---

thededemo@Demo-VH:~/all app folder/Code/c-c++/lap_trinh_mang/fork\$

Kết quả http_server

3. Bài tập time_server

```
fork > C time_server.c > main()
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/socket.h>
5 #include <arpa/inet.h>
6 #include <unistd.h>
7 #include <time.h>
8 #include <sys/wait.h>
9 #include <signal.h>
10
11 #define PORT 9001
12
13 void sigchld_handler(int s) {
14     while(waitpid(-1, NULL, WNOHANG) > 0);
15 }
16
17 void get_formatted_time(char *format, char *output) {
18     time_t rawtime;
19     struct tm *info;
20     time(&rawtime);
21     info = localtime(&rawtime);
22
23     if (strcmp(format, "dd/mm/yyyy") == 0) {
24         strftime(output, 50, "%d/%m/%Y", info);
25     } else if (strcmp(format, "dd/mm/yy") == 0) {
26         strftime(output, 50, "%d/%m/%y", info);
27     } else if (strcmp(format, "mm/dd/yyyy") == 0) {
28         strftime(output, 50, "%m/%d/%Y", info);
29     } else if (strcmp(format, "mm/dd/yy") == 0) {
30         strftime(output, 50, "%m/%d/%y", info);
31     } else {
32         strcpy(output, "INVALID_FORMAT");
33     }
34 }
```

Code time_server 1

```

fork > C time_server.c > main()
36 int main() {
37     int listener = socket(AF_INET, SOCK_STREAM, 0);
38     struct sockaddr_in addr = {0};
39     addr.sin_family = AF_INET;
40     addr.sin_addr.s_addr = htonl(INADDR_ANY);
41     addr.sin_port = htons(PORT);
42
43     if (bind(listener, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
44         perror("bind() failed");
45         return 1;
46     }
47     listen(listener, 10);
48     printf("Time Server đang chạy trên cổng %d sử dụng Multiprocessing...\n", PORT);
49
50     struct sigaction sa;
51     sa.sa_handler = sigchld_handler;
52     sigemptyset(&sa.sa_mask);
53     sa.sa_flags = SA_RESTART;
54     sigaction(SIGCHLD, &sa, NULL);
55
56     while (1) {
57         int client_fd = accept(listener, NULL, NULL);
58         if (client_fd < 0) continue;
59
60         printf("Chấp nhận kết nối mới. Đang tạo tiến trình con...\n");
61
62         if (fork() == 0) {
63             close(listener);
64             char buf[256];
65             while (1) {
66                 int ret = recv(client_fd, buf, sizeof(buf) - 1, 0);
67                 if (ret <= 0) break;
68
69                 buf[ret] = '\0';

```

Code time server 2

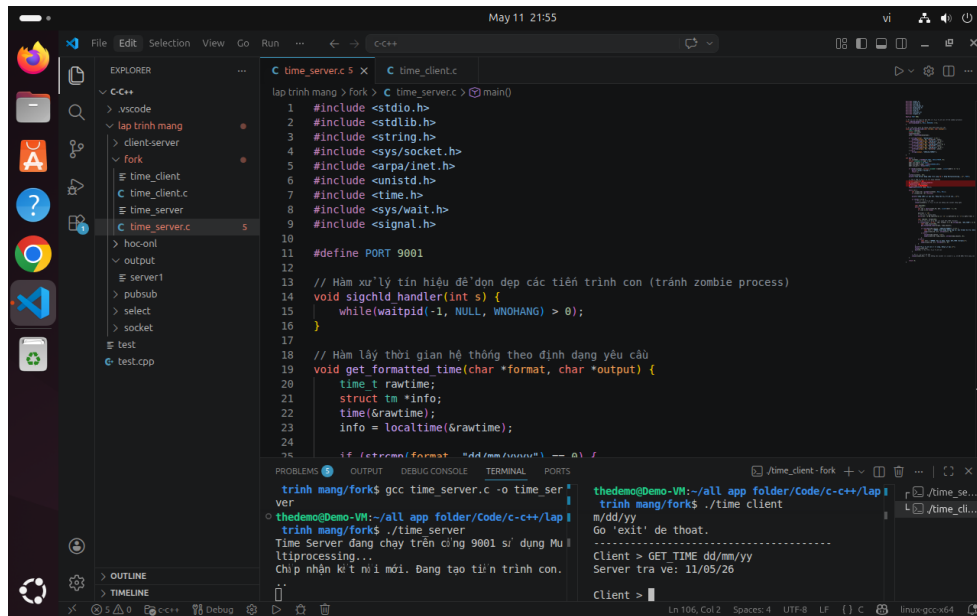
```

fork > C time_server.c > main()
69     buf[ret] = '\0';
70     while(ret > 0 && (buf[ret-1] == '\n' || buf[ret-1] == '\r')) buf[--ret] = '\0';
71
72     char cmd[20], format[50];
73     if (sscanf(buf, "%s %s", cmd, format) == 2 && strcmp(cmd, "GET_TIME") == 0) {
74         char time_result[50];
75         get_formatted_time(format, time_result);
76
77         if (strcmp(time_result, "INVALID_FORMAT") == 0) {
78             char *err = "ERROR: Định dạng không hợp lệ. Các format hỗ trợ: dd/mm/yyyy, d
79             send(client_fd, err, strlen(err), 0);
80         } else {
81             strcat(time_result, "\n");
82             send(client_fd, time_result, strlen(time_result), 0);
83         }
84     } else {
85         char *err = "ERROR: Sai cú pháp. Dùng: GET_TIME [format]\n";
86         send(client_fd, err, strlen(err), 0);
87     }
88
89     printf("Tiến trình con xử lý xong, đóng kết nối.\n");
90     close(client_fd);
91     exit(0);
92 }
93
94     close(client_fd);
95 }
96
97     return 0;
98 }

```

Code time_server 3

- **Kết quả:**



The screenshot shows a Visual Studio Code editor with a C++ project. The Explorer sidebar on the left shows the project structure: `.vscode`, `lap_trinh_mang`, `client-server`, `fork`, `time_client.c`, `time_server.c`, `hoconl`, `output`, `serverl`, `pubsub`, `select`, `socket`, `test`, and `test.cpp`. The main editor displays the source code for `time_server.c` and `time_client.c`. The `time_server.c` code includes headers for `stdio.h`, `stdlib.h`, `string.h`, `sys/socket.h`, `arpa/inet.h`, `unistd.h`, `time.h`, `sys/wait.h`, and `signal.h`. It defines a port of 9001 and implements a signal handler to prevent zombie processes. The `time_client.c` code is partially visible, showing a `main` function that sends a `GET TIME` request to the server. The bottom panel shows the terminal output, where the server is running on port 9001 and the client is connected. The client sends the request, and the server responds with the current date and time: `Server tra ve: 11/05/26`.

```
lap_trinh_mang > fork > C time_server.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/socket.h>
5  #include <arpa/inet.h>
6  #include <unistd.h>
7  #include <time.h>
8  #include <sys/wait.h>
9  #include <signal.h>
10
11 #define PORT 9001
12
13 // Hàm xử lý tín hiệu để dọn dẹp các tiến trình con (tránh zombie process)
14 void sigchld_handler(int s) {
15     while(waitpid(-1, NULL, WNOHANG) > 0);
16 }
17
18 // Hàm lấy thời gian hệ thống theo định dạng yêu cầu
19 void get_formatted_time(char *format, char *output) {
20     time_t rawtime;
21     struct tm *info;
22     time(&rawtime);
23     info = localtime(&rawtime);
24
25     if (strftime(output, sizeof(output), format, info) == 0) {
26         printf("Error: strftime failed\n");
27     }
28 }
```

```
thetdemo@Deno-VM:~/all_app_folder/Code/c-c++/lap_trinh_mang/fork$ gcc time_server.c -o time_server
thetdemo@Deno-VM:~/all_app_folder/Code/c-c++/lap_trinh_mang/fork$ ./time_server
Time Server đang chạy trên cổng 9001 sử dụng Multi-processing...
Chấp nhận kết nối mới. Đang tạo tiến trình con.

thetdemo@Deno-VM:~/all_app_folder/Code/c-c++/lap_trinh_mang/fork$ ./time_client
Client > GET TIME dd/mm/yy
Server tra ve: 11/05/26
Client >
```